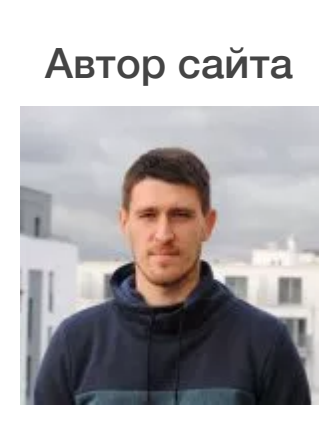
Language

🇷🇺 🇬🇧

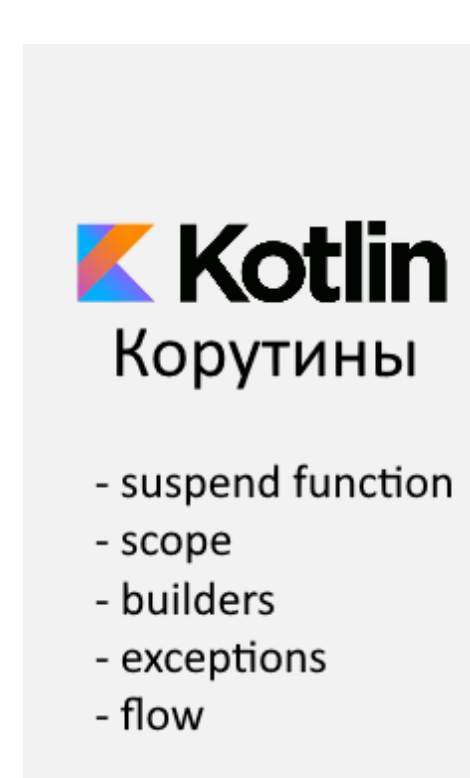
Автор сайта



Дмитрий Виноградов

Подробнее можно посмотреть или почитать.

Никакие другие люди не имеют к этому сайту никакого отношения и просто занимаются плагиатом.



Социальные сети

В канале я публикую ссылки на интересные и полезные статьи по Android

В чате можно обсудить вопросы и проблемы, возникающие при разработке

Группа ВКонтакте

StartAndroid

15 084 подписчика

Подписаться

Поддержка проекта

Яндекс
410011180491924

WebMoney
R248743991365
Z551306702056

Paypal

Урок 3. Дополнительные возможности

21 июня 2021

В этом уроке рассмотрим некоторые дополнительные возможности даггера при создании объектов: Lazy, Provider, Named, Qualifier, IntoSet, ElementsIntoSet, IntoMap.

Lazy

Ленивое создание объекта.

Рассмотрим пример:

```
class MainActivity : AppCompatActivity() {

   lateinit var networkUtils: NetworkUtils

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContentView(R.layout.activity_main)

       networkUtils = (application as App).appComponent.getNetworkUtils()
   }

}
```

Здесь компонент просто создает для нас объект NetworkUtils.

Но что если этот объект нужен нам не всегда, а только в некоторых случаях. Например, когда пользователь жмет на экране определенную кнопку. Т.е. может быть так, что пользователь уйдет с экрана так и не нажав эту кнопку. Тогда получится что мы зря создавали объект NetworkUtils.

В таких случаях обычно используется ленивое создание объекта. Даггер умеет это делать из коробки с помощью Lazy обертки.

```
class MainActivity : AppCompatActivity() {

   lateinit var networkUtilsLazy: Lazy<NetworkUtils>

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContentView(R.layout.activity_main)

       networkUtilsLazy = (application as App).appComponent.getNetworkUtils()
   }

}
```

Оборачиваем NetworkUtils в Lazy.

То же самое делаем на стороне компонента:

```
@Component(modules = [...])
interface AppComponent {
   fun getNetworkUtils(): Lazy<NetworkUtils>
}
```

Обратите внимание, что в котлине есть своя обертка Lazy. И по умолчанию будет использоваться именно она. Это неправильно. Убедитесь, что вы использовали Lazy именно от даггера.

У вас должен появиться такой импорт:

```
import dagger.Lazy
```

Либо можно сразу в коде писать полное имя с пакетом dagger:

```
lateinit var networkUtilsLazy: dagger.Lazy<NetworkUtils>
```

Итак у нас есть networkUtilsLazy: Lazy<NetworkUtils>. Он умеет создавать объект NetworkUtils.

Когда пользователь жмет заветную кнопку, мы понимаем, что нам нужен объект NetworkUtils. Мы используем метод Lazy.get, чтобы создать его:

```
val networkUtils = networkUtilsLazy.get()
```

Все последующие вызовы get уже не будут создавать новые NetworkUtils объекты, а будут возвращать тот, который был создан при первом вызове get.

## Provider

Полностью аналогичен Lazy, но при каждом вызове get он будет создавать новый объект. Т.е. это фактически фабрика объекта.

Named

Иногда бывает необходимость создавать несколько объектов одного класса, но с разными входными данными.

Рассмотрим пример. У нас есть некий класс для работы с сервером:

```
class ServerApi(private val host: String) {
   // ...
}
```

На вход он требует указать ему имя хоста. У нас есть два варианта хоста: prod.server.com и dev.server.com.

Создаем два разных ServerApi в модуле даггера:

```
@Provides
fun provideServerApiProd(): ServerApi {
   return ServerApi("prod.server.com")
}

@Provides
fun provideServerApiDev(): ServerApi {
   return ServerApi("dev.server.com")
}
```

Один объект работает с prod сервером, другой - с dev.

Прописываем в компоненте метод для получения ServerApi

```
@Component(modules = [...])
interface AppComponent {

   fun getServerApiProd(): ServerApi

}
```

Уже тут возникает вопрос: а какой именно объект ServerApi из двух возможных создаст нам даггер.

Если мы попытаемся запустить проект, то получим ошибку:

ServerApi is bound multiple times

Даггер видит, что есть два способа создать объект ServerApi. И он не знает, какой именно надо использовать. Решается это аннотацией Named.

Добавим Named к Provides методам:

```
@Named("prod")
@Provides
fun provideServerApiProd(): ServerApi {
   return ServerApi("prod.server.com")
}

@Named("dev")
@Provides
fun provideServerApiDev(): ServerApi {
   return ServerApi("dev.server.com")
}
```

Теперь даггер может отличить эти объекты.

А мы сможем указать, какой из них нам нужен. Для этого мы также используем Named в компоненте:

```
@Component(modules = [...])
interface AppComponent {

   @Named("prod")
   fun getServerApiProd(): ServerApi
}
```

Просим у компонента ServerApi, который prod.

На стороне получателя уже никаких аннотаций указывать не надо:

```
class MainActivity : AppCompatActivity() {

   lateinit var serverApi: ServerApi

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContentView(R.layout.activity_main)

       serverApi = (application as App).appComponent.getServerApiProd()
   }

}
```

Если же мы в компоненте используем не get, а inject метод, то в компоненте аннотации не нужны:

```
@Component(modules = [StorageModule::class, NetworkModule::class, MainModule::class])
interface AppComponent {

   fun injectMainActivity(mainActivity: MainActivity)

}
```

А вот на стороне получателя уже надо будет указать Named, чтобы даггер знал, какой именно ServerApi инджектить:

```
class MainActivity : AppCompatActivity() {

   @Inject
   @Named("prod")
   lateinit var serverApi: ServerApi

   override fun onCreate(savedInstanceState: Bundle?) {
       super.onCreate(savedInstanceState)
       setContentView(R.layout.activity_main)

       (application as App).appComponent.injectMainActivity(this)
   }

}
```

Qualifier

Вместо только что рассмотренной нами аннотации @Named с текстовым значением мы можем создавать свои аннотации. Для этого необходимо использовать Qualifier.

Создадим две аннотации: Prod и Dev:

```
@Qualifier
@Retention(AnnotationRetention.RUNTIME)
annotation class Prod
```

```
@Qualifier
@Retention(AnnotationRetention.RUNTIME)
annotation class Dev
```

И используем их вместо аннотации Named:

```
@Prod
@Provides
fun provideServerApiProd(): ServerApi {
   return ServerApi("prod.server.com")
}

@Dev
@Provides
fun provideServerApiDev(): ServerApi {
   return ServerApi("dev.server.com")
}
```

Они так же, как и Named, помогут даггеру различать два способа создания объекта ServerApi.

И мы, соответственно, используем одну из них, чтобы сказать компоненту, какой именно ServerApi нам нужен:

```
@Component(modules = [...])
interface AppComponent {

   @Prod
   fun getServerApiProd(): ServerApi

}
```

В Qualifier аннотации, которые мы создаем, мы можем добавлять аргументы. Например, у нас есть два Prod сервера: prod1.server.com и prod2.server.com. И мы хотим под каждый из них создавать свой ServerApi.

Можно конечно создать аннотации Prod1 и Prod2. А можно использовать одну аннотацию Prod, но с аргументом.

Добавляем String аргумент к аннотации Prod:

```
@Qualifier
@Retention(AnnotationRetention.RUNTIME)
annotation class Prod(val value: String = "")
```

И используем для создания двух разных объектов ServerApi:

```
@Prod("1")
@Provides
fun provideServerApiProd1(): ServerApi {
   return ServerApi("prod1.server.com")
}

@Prod("2")
@Provides
fun provideServerApiProd2(): ServerApi {
   return ServerApi("prod2.server.com")
}
```

В компоненте также указываем, какой сервер хотим получить:

```
@Component(modules = [...])
interface AppComponent {

   @Prod("2")
   fun getServerApiProd(): ServerApi

}
```

IntoSet

Если нам необходимо от компонента получить несколько однотипных объектов, мы можем запросить их сразу как Set.

Рассмотрим на примере. Пусть у нас есть какое-то глобальное событие Event

```
class Event(...)
```

Мы хотим пропускать его через несколько обработчиков EventHandler:

```
interface EventHandler {
   fun handle(event: Event)
}
```

Такими обработчиками могут быть, например, аналитика и логи:

```
class Analytics: EventHandler {
   override fun handle(event: Event) {
       // ...
   }
}
```

```
class Logger: EventHandler {
   override fun handle(event: Event) {
       // ...
   }
}
```

Было бы удобно сразу получить от даггера готовый набор обработчиков, а не по одному. Даггер может собрать их для нас в коллекцию Set.

Для этого используем аннотацию IntoSet с Provides методами обработчиков:

```
@IntoSet
@Provides
fun provideAnalytics(): EventHandler {
   return Analytics()
}

@IntoSet
@Provides
fun provideLogger(): EventHandler {
   return Logger()
}
```

Обратите внимание, что тип возвращаемого значения у этих методов - EventHandler. Теперь даггер сможет собрать коллекцию Set<EventHandler>, состоящую из двух этих объектов.

А получить ее мы можем, прописав в компоненте соответствующий метод:

```
@Component(modules = [...])
interface AppComponent {

   fun getEventHandlers(): Set<EventHandler>

}
```

ElementsIntoSet

Эта аннотация работает примерно так же, как и IntoSet, но добавляет в коллекцию не один элемент, а коллекцию элементов.

В качестве примера давайте продолжим тему с обработчиками событий. Например, у нас есть несколько EventHandlers, которые связаны с базой данных. Они нам доступны в DatabaseHelper:

```
fun DatabaseHelper.getEventHandlers(): Set<EventHandler>
```

Мы хотим добавить эти обработчики к общему списку, где уже есть Analytics и Logger. Для этого используем ElementsIntoSet с методом @Provides:

```
@ElementsIntoSet
@Provides
fun provideDatabaseEventHandlers(databaseHelper: DatabaseHelper): Set<EventHandler> {
   return databaseHelper.getEventHandlers()
}
```

В итоге, когда даггер будет собирать нам коллекцию Set<EventHandler>, он добавит туда все элементы из databaseHelper.getEventHandlers().

## IntoMap

Эта аннотация аналогична IntoSet, но собирает объекты не в Set, а в Map. А от нас требуется указать ключ, с которым объект будет помещен в Map.

Рассмотрим снова пример с EventHanlder. Мы хотим получать их от компонента не по одному, а все сразу в коллекции Map.

Для этого вместе в Provides методах используем IntoMap. А чтобы указать ключ, используем @StringKey.

```
@IntoMap
@StringKey("analytics")
@Provides
fun provideAnalytics(): EventHandler {
   return Analytics()
}

@IntoMap
@StringKey("logger")
@Provides
fun provideLogger(): EventHandler {
   return Logger()
}
```

Используя ключ типа String, мы тем самым определили тип ключа в Map. Даггер соберет объекты в Map<String, EventHandler>

Чтобы мы могли получить их, описываем метод в компоненте:

```
@Component(modules = [...])
interface AppComponent {

   fun getEventHandlers(): Map<String, EventHandler>

}
```

Кроме String, даггер также предоставляет аналогичные аннотации для Map-ключей с типами Long, Int и Class.

Если этого недостаточно, можно создать свою аннотацию, которая задаст нужный нам тип ключа.

Допустим, вместо текста в качестве ключа мы хотим использовать enum. Создаем enum класс:

```
enum class EventHandlerType {
   ANALYTICS, LOGGER
}
```

Создаем свою аннотацию EventHandlerKey с помощью MapKey:

```
@MapKey
@Retention(AnnotationRetention.RUNTIME)
annotation class EventHandlerKey(val value: EventHandlerType)
```

Указываем, что требуется значение с типом EventHandlerType. Это и будет ключом в коллекции Map.

Используем созданную аннотацию с Provides методами:

```
@IntoMap
@EventHandlerKey(EventHandlerType.ANALYTICS)
@Provides
fun provideAnalytics(): EventHandler {
   return Analytics()
}

@IntoMap
@EventHandlerKey(EventHandlerType.LOGGER)
@Provides
fun provideLogger(): EventHandler {
   return Logger()
}
```

И в компоненте просим даггер дать нам Map с нашим созданным типом ключа - Map<EventHandlerType, EventHandler>:

```
@Component(modules = [...])
interface AppComponent {

   fun getEventHandlers(): Map<EventHandlerType, EventHandler>

}
```

Назад Вперёд

Комментарии

# **IntoMap** — **Анна** 22.06.2021 16:39
А как быть с IntoMap, если работаем с @inject?
Ответить

# **RE: IntoMap** — **Dmitry Vinogradov** 22.06.2021 21:52
Такое сделать не получится, к сожалению
https://github.com/google/dagger/issues/510
Ответить

# **RE: Урок 3. Дополнительные возможности** — **Иван** 01.10.2021 18:26
@Component(modules = [...])
interface AppComponent {

fun getEventHanLDers(): Set

}
Ответить

# **RE: Урок 3. Дополнительные возможности** — **Dmitry Vinogradov** 04.10.2021 14:20
Пофиксил, спасибо!
Ответить

# **Provider import** — **Сергей** 23.11.2021 03:04
Из описания предположил, что у Provider должен быть "import dagger.Provider", но оказалось, что для него должен быть "import javax.inject.Provider". Если это конечно тот, что описан в статье.
Ответить

# **RE: Provider import** — **Dmitry Vinogradov** 02.12.2021 13:25
Все верно
javax.inject.Provider
Ответить

# **IntoMap и Inject** — **Даниил** 03.04.2023 09:23
Выше написали, что нельзя юзать IntoMap с Inject. А как тогда работает этот код? Здесь формируется Map из viewModels в Inject-конструкторе:

```
* Фабрика, нужная для создания ViewModelProvider.Factory любой ViewModel, у которой @Inject constructor
*
* Пример использования ViewModelFactory во фрагментах:
*
* @Inject
* lateinit var viewModelFactory: ViewModelProvider.Factory
*
* private val viewModel: SomeViewModel by viewModels { viewModelFactory }
*
*/
class DaggerViewModelFactory @Inject constructor(
private val viewModelsMap: Map
) : ViewModelProvider.Factory {

override fun create(modelClass: Class): T {
val creator = viewModelsMap[modelClass] ?: viewModelsMap.asIterable().firstOrNull {
modelClass.isAssignableFrom(it.key)
}?.value ?: throw IllegalArgumentException("unknown model class $modelClass")
return try {
creator.get() as T
} catch (e: Exception) {
throw RuntimeException(e)
}
}
}

/**
* Использовать для всех ViewModel у которых @Inject constructor
*
* Пример использования:
* @Module
* abstract class BankViewModelModule {
* @Binds
* @IntoMap
* @ViewModelKey(SomeViewModel::class)
* abstract fun bindSomeViewModel(viewModel: SomeViewModel): ViewModel
* }
*
*/
@Target(
AnnotationTarget.FUNCTION,
AnnotationTarget.PROPERTY_GETTER,
AnnotationTarget.PROPERTY_SETTER
)
@MapKey
annotation class ViewModelKey(val value: KClass)

-----------------------------------------

/**
* Переиспользуемый модуль для инициализации [DaggerViewModelFactory]
* Должен использоваться во всех компонентах, где есть View Model
*/
@Module
abstract class ViewModelFactoryModule {
@Binds
abstract fun bindViewModelFactory(viewModelFactory: DaggerViewModelFactory): ViewModelProvider.Factory
}

---------------------------------
Вот так выглядит ViewModelModule:

@Module(includes = [ViewModelFactoryModule::class])
abstract class ViewModelModule {

@Binds
@IntoMap
@ViewModelKey(FirstViewModel::class)
abstract fun bindFirstViewModel(viewModel: FirstViewModel): ViewModel

@Binds
@IntoMap
@ViewModelKey(SecondViewModel::class)
abstract fun bindSecondViewModel(viewModel: SecondViewModel): ViewModel

и тд.. все ViewModel
}
```

Ответить

# **@JvmSuppressWildcards** — **Даниил** 03.04.2023 09:37
Ответ на мой коммент выше - походу все дело в аннотации @JvmSuppressWildcards. Благодаря ей, становится возможным юзать юзать IntoMap с Inject
Ответить

# **@JvmSuppressWildcards** — **Даниил** 03.04.2023 09:44
Почему-то объявление выше для DaggerViewModelFactory вставилось не полностью.

```
class DaggerViewModelFactory @Inject constructor(
private val viewModelsMap: Map
) : ViewModelProvider.Factory {
....
}
```

Вот здесь видно, что используется @JvmSuppressWildcards с Map, что позволяет юзать IntoMap с Inject
Ответить

# **Походу при вставке обрезается** — **Даниил** 03.04.2023 09:46
При вставке обрезается почему-то здесь полное объявление класса, выше опять неверно вставилось
private val viewModelsMap: Map
Ответить

# **RE: Урок 3. Дополнительные возможности** — **Даниил** 03.04.2023 09:46
Map
Ответить

# **@JvmSuppressWildcards** — **Даниил** 03.04.2023 09:51
Видимо сайт путает мои скобки дженериков с тегами. Вот короче ответ:
https://github.com/google/dagger/issues/1822
Ответить

Обновить список комментариев

RSS лента комментариев этой записи

Добавить комментарий

Тема

😀😁🙂😉😎😐🙄😡😕😲😵😳🤢😮😴😋😊😌

Отправить

JComments

© Start Android, 2011-2020. Все права защищены, копирование материалов возможно только с разрешения автора

При возникновении каких-либо вопросов, используйте форму обратной связи

Bootstrap is a front-end framework of Twitter, Inc. Code licensed under MIT License.
Font Awesome font licensed under SIL OFL 1.1.